



11. 표준 라이브러리와 순차 컨테이너

Heesung Oh

<https://github.com/3dapi>



- STL 방식의 핵심 요소
- 컨테이너, 반복자, 알고리즘, 함수 객체
- 컨테이너 공통 기능
- 반복자·포인터·참조의 무효화
- `std::array`, `std::vector`, `std::deque`
- `std::list`, `std::forward_list`
- `std::string`
- 순차 컨테이너 선택 기준

컨테이너

↓ 반복자 범위

알고리즘

↓ 함수 객체 또는 람다

처리 결과





- 자주 사용하는 자료 구조와 알고리즘 제공
- 문자열, 입출력, 메모리 관리, 시간, 난수, 파일 시스템, 스레드 포함
- 대부분 std 이름 공간에 정의
- 필요한 기능은 헤더를 포함하여 사용

구성 요소	대표 예
컨테이너	<code>std::vector</code> , <code>std::map</code>
알고리즘	<code>std::find</code> , <code>std::sort</code>
문자열	<code>std::string</code>
메모리 관리	<code>std::unique_ptr</code>
입출력	<code>std::cout</code> , <code>std::ifstream</code>
유틸리티	<code>std::move</code> , <code>std::tuple</code>

```
#include <vector>
#include <algorithm>
#include <memory>
#include <string>
```



- STL은 컨테이너, 반복자, 알고리즘, 함수 객체를 템플릿으로 연결한 설계에서 출발
- 현재 표준 라이브러리는 STL 계열 기능보다 넓은 범위
- `iostream`, `thread`, `filesystem`도 표준 라이브러리의 일부

구분	의미	대표 예
컨테이너	여러 원소 저장	<code>vector</code> , <code>list</code>
반복자	원소 위치와 범위 표현	<code>begin()</code> , <code>end()</code>
알고리즘	범위에 작업 적용	<code>sort</code> , <code>find</code>
함수 객체	동작을 객체로 전달	<code>less</code> , 람다
표준 라이브러리 전체	C++ 공통 라이브러리	STL + 입출력 + 스레드 등





- 여러 원소를 저장하고 원소의 수명과 배치를 관리
- 저장되는 타입을 원소 타입이라고 함
- 컨테이너는 원소를 값으로 소유
- 포인터를 저장하는 경우 가리키는 객체의 수명은 별도 관리 필요

```
std::vector<int> scores;  
std::list<std::string> names;
```

범주	특징	대표 타입
순차 컨테이너	위치 순서에 따라 배치	array, vector, deque, list
연관 컨테이너	정렬된 키로 탐색	set, map
비정렬 연관 컨테이너	해시로 탐색	unordered_set, unordered_map
컨테이너 어댑터	제한된 인터페이스 제공	stack, queue





- 컨테이너의 원소 위치를 나타내는 객체
- *로 원소 접근
- ++로 다음 위치 이동
- begin()은 첫 원소 위치
- end()는 마지막 원소 바로 뒤 위치

```
std::vector<int> values{10, 20, 30};
```

```
auto iterator = values.begin();
```

```
std::cout << *iterator << '\n';
```

```
++iterator;
```

```
std::cout << *iterator << '\n';
```

[10]	[20]	[30]	[끝 위치]
↑			↑
begin()			end()



- 반복자 범위는 [first, last) 형태
- first는 포함
- last는 포함하지 않음
- 빈 범위는 first == last
- 알고리즘은 필요한 반복자 능력을 요구

반복자 범주	주요 연산	대표 컨테이너
전진 반복자	++	forward_list
양방향 반복자	++, --	list
임의 접근 반복자	+, -, [], 비교	vector, deque, array
연속 반복자	임의 접근 + 연속 메모리	array, vector, string

std::find() → 전진 순회 가능하면 사용

std::sort() → 임의 접근 반복자 필요



- 반복자 범위를 받아 작업을 수행하는 함수 템플릿
- 검색, 정렬, 복사, 변환, 집계 기능 제공
- 주로 `<algorithm>`, `<numeric>` 헤더에 정의
- 컨테이너를 소유하지 않고 범위의 원소에 작업 적용

```
#include <algorithm>
#include <numeric>
#include <vector>

std::vector<int> values{5, 2, 8, 2, 1};

auto position = std::find(values.begin(), values.end(), 8);
std::sort(values.begin(), values.end());
int total = std::accumulate(values.begin(), values.end(), 0);

if (position != values.end())
{
    std::cout << "값을 찾음\n";
}
```




- 함수 호출 연산자 `operator()`를 제공하는 객체
- 알고리즘에 비교, 판정, 변환 규칙 전달
- 람다 표현식도 호출 가능한 객체
- 상태를 가진 판정 객체도 만들 수 있음

```
std::vector<int> values{5, 1, 9, 3};  
std::sort(values.begin(), values.end(), std::greater<int>{});  
std::sort(values.begin(), values.end(), [](int left, int right) { return left > right; });
```

사용 위치	전달하는 의미
<code>sort()</code> 비교자	정렬 순서
<code>find_if()</code> 조건	검색 조건
<code>count_if()</code> 조건	개수 계산 조건



- 컨테이너는 원소 저장과 반복자 제공 담당
- 알고리즘은 반복자 범위에 작업 적용
- 같은 알고리즘을 여러 컨테이너에 적용 가능
- 반복자 능력이 부족하면 특정 알고리즘 사용 불가
- 인터페이스의 공통성, 저장 구조의 차이를 함께 이해

```
std::vector<int> dynamicValues{4, 1, 3, 2};  
std::array<int, 4> fixedValues{8, 5, 7, 6};  
  
std::sort(dynamicValues.begin(), dynamicValues.end());  
std::sort(fixedValues.begin(), fixedValues.end());  
  
std::list<int> levels{3, 1, 4, 2};  
  
// std::sort(levels.begin(), levels.end()); // 오류  
levels.sort();
```



- 표준 컨테이너는 비슷한 이름의 멤버 제공
- 모든 컨테이너가 완전히 같은 기능을 제공하지는 않음
- 저장 구조에 따라 접근, 삽입, 제거 방식이 달라짐

기능	대표 멤버
크기 확인	<code>size()</code> , <code>empty()</code>
원소 접근	<code>front()</code> , <code>back()</code> , <code>operator[]</code> , <code>at()</code>
반복자	<code>begin()</code> , <code>end()</code> , <code>cbegin()</code> , <code>cend()</code>
삽입	<code>push_back()</code> , <code>insert()</code> , <code>emplace()</code>
제거	<code>pop_back()</code> , <code>erase()</code> , <code>clear()</code>
• 고정 크기 컨테이너 → 원소 추가 함수 없음 • 단방향 리스트 → 뒤쪽 접근과 역방향 반복자 없음	





- 컨테이너는 원소와 반복자에 대한 타입 별칭 제공
- `size()`는 현재 원소 수 반환
- `empty()`는 원소가 없는지 확인
- 빈 컨테이너에서 `front()`, `back()` 호출 금지

```
using Container = std::vector<int>;  
Container::value_type value = 10;  
Container::size_type count = 0;  
Container::iterator iterator;  
Container::const_iterator const_iterator;
```

타입 별칭	의미
<code>value_type</code>	원소 타입
<code>size_type</code>	원소 수 타입
<code>difference_type</code>	반복자 거리 타입
<code>iterator</code>	수정 가능한 반복자
<code>const_iterator</code>	읽기 전용 반복자



- 저장 구조에 따라 제공되는 접근 함수가 다름
- `operator[]`는 범위 검사 없음
- `at()`은 범위 오류 시 예외 발생
- `data()`는 연속 저장 영역의 시작 주소 반환

함수	의미	주요 지원 컨테이너
<code>front()</code>	첫 원소	모든 순차 컨테이너
<code>back()</code>	마지막 원소	<code>array</code> , <code>vector</code> , <code>deque</code> , <code>list</code> , <code>string</code>
<code>operator[]</code>	범위 검사 없는 인덱스 접근	<code>array</code> , <code>vector</code> , <code>deque</code> , <code>string</code>
<code>at()</code>	범위 검사 인덱스 접근	<code>array</code> , <code>vector</code> , <code>deque</code> , <code>string</code>
<code>data()</code>	연속 저장 주소	<code>array</code> , <code>vector</code> , <code>string</code>

```
std::vector<int> values{10, 20, 30};  
int value = values.at(1);
```





- 가변 크기 컨테이너는 저장 구조에 맞는 삽입과 제거 제공
- `push_back()`은 끝에 원소 추가
- `pop_back()`은 끝 원소 제거
- `insert()`는 삽입 위치를 반복자로 받음
- `erase()`는 제거한 원소 뒤 위치를 반복자로 반환

```
std::vector<int> values;
```

```
values.push_back(10);  
values.push_back(20);  
values.insert(values.begin() + 1, 15);  
values.erase(values.begin());  
values.pop_back();
```

```
for (auto iterator = values.begin();  
     iterator != values.end(); )  
{  
    if (*iterator < 0)  
    {  
        iterator = values.erase(iterator);  
    }  
    else  
    {  
        ++iterator;  
    }  
}
```





11.2.4 begin과 end

- begin()과 end()는 반복자 범위 제공
- const 컨테이너에서는 읽기 전용 반복자 반환
- cbegin(), cend()는 항상 읽기 전용 반복자 반환
- rbegin(), rend()는 역방향 순회에 사용
- 원소를 복사할 필요가 없으면 참조 사용
- 읽기만 하면 range for + const 참조 사용

```
std::vector<int> values{10, 20, 30};  
  
for (auto itr = values.begin(); iterator != values.end(); ++itr)  
{  
    *itr *= 2;  
}  
  
for (const std::string& name : names)  
{  
    std::cout << name << '\n';  
}
```



- 컨테이너를 복사하면 원소도 복사
- 복사본은 별도 저장 공간을 소유
- 컨테이너를 이동하면 내부 저장 자원을 이전할 수 있음
- 이동 후 원본 컨테이너는 유효하지만 값은 특정 상태로 보장되지 않을 수 있음

```
std::vector<std::string> original{"A", "B", "C"};  
std::vector<std::string> copied = original;
```

```
std::vector<std::string> moved = std::move(original);
```

```
std::vector<std::unique_ptr<int>> values;  
values.push_back(std::make_unique<int>(10));
```

```
// auto copied = values; // 원소를 복사할 수 없어 오류
```




- 컨테이너 변경 후 기존 위치 객체를 사용할 수 없게 되는 상태
- 무효화된 반복자, 포인터, 참조는 자동으로 nullptr이 되지 않음
- 사용하면 미정의 동작 가능
- 재할당, 원소 이동, 제거가 주요 원인

```
std::vector<int> values{10, 20, 30};
```

```
auto iterator = values.begin();
```

```
int* pointer = values.data();
```

```
int& reference = values.front();
```

```
values.reserve(values.capacity() + 1);
```

```
// 재할당이 발생하면 iterator, pointer, reference 사용 불가
```



컨테이너와 연산	무효화 범위
std::array 원소 값 변경	반복자, 포인터, 참조 유지
std::vector 재할당	전체 무효화
std::vector 중간 삽입·제거	삽입·제거 위치부터 끝까지
std::deque 양 끝 삽입	모든 반복자 무효, 기존 원소 참조와 포인터 유지
std::deque 중간 삽입	모든 반복자, 포인터, 참조 무효
std::list 삽입	기존 반복자, 포인터, 참조 유지
std::list 제거	제거된 원소 위치만 무효
std::string 재할당	전체 무효화

- 무효화 여부는 컨테이너, 연산, 위치, 재할당 여부를 함께 확인
- 컨테이너 변경 뒤에는 반환 반복자나 새 반복자를 사용



- `std::array<T, N>`은 고정 크기 순차 컨테이너
- 원소 타입과 크기를 타입에 포함
- 내부 원소는 C 배열처럼 연속 메모리에 저장
- 반복자, 대입, 비교, 크기 확인 제공

```
#include <array>
std::array<int, 4> values{10, 20, 30, 40};
```

특징	의미
고정 크기	실행 중 원소 수 변경 불가
연속 메모리	C 배열처럼 저장
객체로 전달	크기 정보 유지
대입 가능	같은 타입끼리 전체 복사
반복자 지원	표준 알고리즘 적용 가능





11.3.1 C 배열과 std::array

- C 배열은 함수 인수에서 포인터로 변환될 수 있음
- 배열 크기 정보가 사라질 수 있음
- std::array는 하나의 객체
- 크기를 스스로 알고 있음
- 대입과 반환 가능

```
void PrintValues(const int values[], std::size_t size)
{
    for (std::size_t index = 0; index < size; ++index)
    {
        std::cout << values[index] << ' ';
    }
}
```

```
void PrintValues(const std::array<int, 4>& values)
{
    for (int value : values)
    {
        std::cout << value << ' ';
    }
}
```

```
std::array<int, 4> first{1, 2, 3, 4};
std::array<int, 4> second = first;
```





- 원소 수 N은 컴파일 시점에 정해짐
- 객체 수명 동안 크기 변경 불가
- 크기가 다르면 서로 다른 타입
- 기본형 원소를 0으로 초기화하려면 빈 중괄호 사용

```
std::array<int, 3> first{};  
std::array<int, 5> second{};
```

```
std::array<int, 5> values{}; // 모든 원소 0  
std::array<int, 5> items;   // 기본형 원소 값 미정
```

- std::array<T, 0>에서는 front(), back(), operator[] 사용 불가

선언	의미
std::array<int, 3>	원소 3개짜리 타입
std::array<int, 5>	원소 5개짜리 다른 타입
std::array<T, 0>	원소가 없는 유효한 타입



- 연속 반복자 제공
- 표준 알고리즘 적용 가능
- `data()`는 첫 원소 주소 반환
- 같은 원소 타입과 크기이면 대입 가능
- 비교는 원소를 앞에서부터 사전식 비교

```
std::array<int, 5> values{30, 10, 50, 20, 40};

std::sort(values.begin(), values.end());
auto position = std::find(values.begin(), values.end(), 30);

int* data = values.data();
ProcessData(values.data(), values.size());

std::array<int, 3> source{10, 20, 30};
std::array<int, 3> destination{};

destination = source;
```





11.3.5 함수 인수로 전달하기

- 고정된 크기만 받으면 구체 타입 참조 사용
- 여러 크기를 허용하려면 크기를 템플릿 매개변수로 받음
- 연속된 원소 범위만 읽으면 `std::span` 사용 가능
- `std::span`은 원소를 소유하지 않음

```
template <std::size_t N>
int Sum(const std::array<int, N>& values)
{
    int total = 0;

    for (int value : values)
    {
        total += value;
    }

    return total;
}
```

```
#include <span>
int Sum(std::span<const int> values)
{
    int total = 0;

    for (int value : values)
    {
        total += value;
    }

    return total;
}
```



- 원소를 연속 메모리에 저장하는 가변 크기 순차 컨테이너
- 인덱스 접근이 빠름
- 순차 순회에서 캐시 지역성이 좋음
- 가변 크기 순차 컨테이너의 기본 선택으로 적합

```
#include <vector>
```

```
std::vector<int> values{10, 20, 30};
```

특징	의미
가변 크기	원소 추가와 제거 가능
연속 메모리	data() 사용 가능
임의 접근	operator[], at()
뒤쪽 추가 효율	push_back(), emplace_back()
중간 삽입 비용	뒤쪽 원소 이동 가능
재할당	반복자와 참조 무효화 가능





- `std::vector`의 원소는 메모리에 연속 배치
- `data()`는 저장 영역의 시작 주소 반환
- C 함수나 외부 라이브러리에 버퍼 전달 가능
- 크기가 0이면 `data()` 결과를 역참조하면 안 됨

```
std::vector<int> values{10, 20, 30};
```

```
int* first = &values[0];
```

```
int* data = values.data();
```

```
WriteValues(values.data(), values.size());
```

```
long long Sum(const std::vector<int>& values)
{
    long long total = 0;
    for (int value : values)
    {
        total += value;
    }
    return total;
}
```



- 끝에 추가: `push_back()`, `emplace_back()`
- 끝 원소 제거: `pop_back()`
- 중간 삽입: `insert()`
- 중간 제거: `erase()`
- 앞이나 중간 삽입·제거는 원소 이동 비용 발생 가능

```
std::vector<std::string> names;
```

```
names.push_back("Alice");  
names.push_back("Bob");
```

```
if (!names.empty())  
{  
    names.pop_back();  
}
```

```
std::vector<int> values{10, 30};
```

```
values.insert(values.begin() + 1, 20);  
values.erase(values.begin());
```

중간 삽입 → 삽입 위치 뒤 원소 이동
중간 제거 → 제거 위치 뒤 원소 이동



11.4.3 size와 capacity

- `size()`는 현재 생성된 원소 수
- `capacity()`는 재할당 없이 저장할 수 있는 원소 수
- `capacity()`는 원소 수가 아님
- 유효 인덱스는 0부터 `size() - 1`

```
std::vector<int> values;  
std::cout << values.size() << '\n';  
std::cout << values.capacity() << '\n';
```

size = 3, capacity = 5
[10] [20] [30] [빈 공간] [빈 공간]

- 용량 증가 비율은 표준에서 고정하지 않음

항목	의미
<code>size()</code>	실제 원소 수
<code>capacity()</code>	확보된 저장 가능 수
<code>values[index]</code>	$index < size()$ 일 때만 유효



- 원소를 추가할 공간이 부족하면 더 큰 저장 공간 확보
- 기존 원소를 새 공간으로 이동하거나 복사
- 이전 공간 해제
- 재할당이 발생하면 기존 반복자, 포인터, 참조 모두 무효화

```
std::vector<int> values{10, 20, 30};  
int& first = values.front();
```

```
values.push_back(40);
```

// 재할당이 발생했다면 first 사용 불가

```
class Data  
{  
public:  
    Data(const Data&) = default;  
    Data(Data&&) noexcept = default;  
};
```

- 원소 타입의 이동 생성자가 noexcept이면 재할당 과정에서 이동을 선택하기 쉬움
- `size() < capacity()`이면 기존 공간에 원소 생성



- reserve()는 저장 용량 확보
- reserve()는 원소 수를 늘리지 않음
- resize()는 실제 원소 수 변경
- 크기를 줄이면 뒤쪽 원소 소멸

```
std::vector<int> values;  
values.reserve(100);
```

```
std::cout << values.size() << '\n'; // 0  
std::cout << values.capacity() << '\n'; // 100 이상
```

```
std::vector<int> values{10, 20};  
values.resize(5);
```

```
// values  
// 10 20 0 0 0
```

함수	size() 변경	capacity() 확보	새 원소 생성
reserve(n)	아니요	필요하면 증가	아니요
resize(n)	예	필요하면 증가	크기가 늘면 생성



- 중간 삽입과 제거는 재할당이 없어도 원소 이동 발생
- 변경 뒤 같은 위치가 필요하면 반복자를 다시 얻음

연산	조건	무효화 범위
<code>reserve(n)</code>	<code>n > capacity()</code>	전체
<code>reserve(n)</code>	<code>n <= capacity()</code>	없음
<code>push_back()</code> , <code>emplace_back()</code>	재할당 발생	전체
<code>push_back()</code> , <code>emplace_back()</code>	재할당 없음	기존 <code>end()</code>
<code>insert()</code> , <code>emplace()</code>	재할당 발생	전체
<code>insert()</code> , <code>emplace()</code>	재할당 없음	삽입 위치부터 끝까지
<code>erase()</code>	항상	제거 위치부터 끝까지
<code>pop_back()</code>	항상	제거된 원소와 기존 <code>end()</code>
<code>clear()</code>	항상	모든 원소 위치와 기존 <code>end()</code>





11.4.7 push_back과 emplace_back

- push_back()은 원소 타입 객체 하나를 받아 끝에 추가
- emplace_back()은 생성자 인수를 받아 저장 위치에 직접 생성
- emplace_back()이 항상 더 빠른 것은 아님
- 어떤 생성자를 선택하는지 확인 필요

```
struct Item
{
    int id;
    std::string name;
    int count;
};

std::vector<Item> items;
items.push_back({1, "Potion", 10});
```

```
struct Item
{
    int id;
    std::string name;
    int count;

    Item(int itemId, std::string itemName
        , int itemCount)
        : id(itemId), name(std::move(itemName))
        , count(itemCount) { }
};

items.emplace_back(2, "Ether", 5);
```



- double-ended queue에서 온 이름
- 앞과 뒤에서 원소를 효율적으로 추가·제거
- 가변 크기 순차 컨테이너
- 임의 접근 지원
- 전체 원소가 하나의 연속 메모리 블록에 있지는 않음

```
#include <deque>
```

```
std::deque<int> values;
```

```
values.push_back(20);  
values.push_front(10);  
values.push_back(30);
```

```
// values 저장 값  
// 10 20 30
```

특징	의미
<code>push_front()</code>	앞쪽 추가 효율적
<code>push_back()</code>	뒤쪽 추가 효율적
<code>operator[]</code>	임의 접근 지원
<code>data()</code>	제공하지 않음



- 여러 고정 크기 블록을 관리하는 구조
- 구체적인 블록 크기와 내부 배치는 구현마다 다름
- 앞쪽 확장 시 모든 원소를 이동할 필요가 적음
- 전체 원소를 하나의 T*와 크기로 표현할 수 없음

```
std::deque<int> values{10, 20, 30};
```

```
std::cout << values[1] << '\n';
```

```
std::cout << values.at(2) << '\n';
```

```
std::sort(values.begin(), values.end());
```

블록 0	블록 1	블록 2
[10][20]	[30][40]	[50][60]

- 임의 접근 반복자를 제공하므로 std::sort() 사용 가능
- 벡터보다 주소 계산 단계와 캐시 지역성이 불리할 수 있음



- 양 끝 삽입이 많아도 반복자 안정성은 별도 확인 필요

기준	std::vector	std::deque
전체 메모리 연속성	보장	보장하지 않음
뒤쪽 추가	효율적	효율적
앞쪽 추가	원소 이동 필요	효율적
data()	제공	제공하지 않음
캐시 지역성	대체로 좋음	낮을 수 있음
양 끝 삽입 시 참조	재할당 시 무효	기존 원소 참조 유지 가능
양 끝 삽입 시 반복자	재할당 시 무효	모든 반복자 무효

연속 메모리와 외부 API 연동 → **vector**
앞과 뒤 양쪽 작업이 많음 → **deque** 검토





- 원소를 독립된 노드에 저장
- 노드를 포인터로 연결
- 원소가 메모리에 연속 배치되지 않음
- 인덱스 접근 제공하지 않음
- 삽입과 제거에서 다른 원소의 위치가 유지될 수 있음

[값 | 연결] -> [값 | 연결] -> [값 | 연결]

```
std::list<int> values{10, 20, 30};
```

```
auto iterator = values.begin();
```

```
std::advance(iterator, 2);
```

```
std::cout << *iterator << '\n';
```

장점	비용
위치를 알면 노드 연결 변경으로 삽입·제거	특정 위치 탐색은 선형 시간
다른 원소의 참조 안정성	노드별 메모리 할당과 포인터 부가 비용



- `std::list`는 양방향 리스트
- 이전 노드와 다음 노드를 모두 연결
- 양방향 반복자 제공
- `std::forward_list`는 단방향 리스트
- 다음 노드만 연결
- 더 작은 노드 구조를 목표로 함

`std::list`

`null <- [10] <-> [20] <-> [30] -> null`

`std::forward_list`

`[10] -> [20] -> [30] -> null`

기능	list	forward_list
뒤로 이동	가능	불가능
<code>back()</code>	제공	없음
<code>push_back()</code>	제공	없음
<code>size()</code>	제공	없음
역방향 반복자	제공	없음



- `std::list`는 삽입 위치 반복자 사용
- `std::forward_list`는 지정 위치 뒤에 삽입·제거
- 첫 원소 앞의 가상 위치는 `before_begin()`
- 리스트는 노드 연결을 옮기는 `splice()` 제공

```
std::list<int> values{10, 30};  
auto position = std::next(values.begin());  
  
values.insert(position, 20);  
  
std::forward_list<int> values{20, 30};  
  
values.push_front(10);  
values.insert_after(values.begin(), 15);  
values.insert_after(values.before_begin(), 5);  
  
std::list<int> first{10, 20};  
std::list<int> second{30, 40};  
  
first.splice(first.end(), second);
```

- `splice()`는 원소 복사 없이 노드 소유 관계 변경



- 연결 리스트는 원소가 서로 다른 메모리 위치에 있을 수 있음
- 순회할 때 포인터를 따라가므로 캐시 지역성이 낮을 수 있음
- 중간 삽입이 있다는 이유만으로 선택하지 않음
- 삽입 위치를 이미 알고 있는지 확인

```
std::list<int> values{4, 1, 3, 2};  
values.sort();
```

```
values.remove_if(  
    [](int value)  
    {  
        return value < 0;  
    });
```

연결 리스트 선택 조건

- 삽입과 제거 위치를 반복자로 이미 보유
- 삽입과 제거 후 다른 원소의 반복자와 참조 유지 필요
- 여러 리스트 사이에서 원소를 복사하지 않고 이동
- 임의 접근과 연속 메모리가 필요하지 않음



- char 원소로 이루어진 가변 길이 문자열 관리
- 문자 데이터는 연속 메모리에 저장
- 끝에는 C 문자열 호환을 위한 널 문자 관리
- 순차 컨테이너와 유사한 인터페이스 제공

```
#include <string>
```

```
std::string first = "Hello";  
std::string second(5, '*');  
std::string third = first;
```

기능	대표 함수·연산자
길이 확인	size(), length()
연결	+, +=, append()
검색	find(), rfind()
부분 문자열	substr()
삽입·제거	insert(), erase(), replace()
C 문자열 주소	c_str(), data()





- operator+=와 append()로 뒤에 내용 추가
- operator+=는 새 문자열 생성
- 반복 연결에서는 임시 객체와 재할당 발생 가능
- 최종 길이를 예상할 수 있으면 reserve() 검토
- 비교 연산은 문자 코드 기준 사전식 비교

```
std::string message = "Hello";  
message += ", ";  
message += "C++";  
  
std::string firstName = "Ada";  
std::string lastName = "Lovelace";  
std::string fullName = firstName + " " + lastName;  
  
std::string result;  
result.reserve(128);  
  
for (const std::string& word : words)  
{  
    result += word;  
    result += ' ';  
}
```

- 대소문자 무시 비교나 자연어 정렬은 별도 규칙 필요



- find()는 문자열이나 문자를 처음 찾은 위치 반환
- 찾지 못하면 `std::string::npos` 반환
- rfind()는 뒤쪽부터 검색
- substr()은 부분 문자열을 새 객체로 복사
- C++20부터 starts_with(), ends_with() 사용 가능

```
std::string path = "assets/player.png";

std::size_t slash = path.find('/');
std::size_t dot = path.rfind('.');

if (dot != std::string::npos)
{
    std::string extension = path.substr(dot +
1);
}

bool isAsset = path.starts_with("assets/");
bool isPng = path.ends_with(".png");
```





- insert()로 문자열 중간에 내용 삽입
- erase()로 일부 내용 제거
- replace()로 범위를 다른 문자열로 교체
- 한 문자는 push_back(), pop_back()으로 조작
- 중간 변경은 뒤쪽 문자 이동 발생
- 저장 용량 부족 시 재할당 가능

```
std::string text = "Hello World";  
text.insert(5, ",");
```

```
text.erase(5, 1);
```

```
std::string item = "red potion";  
item.replace(0, 3, "blue");
```

```
std::string code = "ABC";  
code.push_back('D');  
code.pop_back();
```





- `std::to_string()`은 숫자를 문자열로 변환
- `std::stoi()` 계열은 문자열을 숫자로 변환
- 변환 실패는 예외로 처리
- 문자열 전체 변환 여부는 변환 위치를 확인
- 예외 없는 저수준 변환은 `std::from_chars()` 사용

```
int score = 1200;
std::string text = std::to_string(score);

std::size_t parsed = 0;
int value = std::stoi("12A", &parsed);

if (parsed != 3)
{
    std::cout << "remaining characters\n";
}

auto r = std::from_chars( text.data(), text.data() + text.size(), value);
```





- `std::string`도 `size()`와 `capacity()`를 가짐
- `reserve()`로 저장 용량 확보 가능
- `data()`와 `c_str()`은 문자 배열 주소 반환
- 문자열 수정으로 재할당이 발생하면 기존 포인터, 반복자, 참조 무효화
- 짧은 문자열 최적화는 구현 세부 사항
- 문자열 크기 밖에 쓰면 안 됨
- 널 종료 규칙을 깨뜨리면 안 됨

```
std::string text = "Hello";
```

```
std::cout << text.size() << '\n';
```

```
std::cout << text.capacity() << '\n';
```

```
text.reserve(256);
```

```
const char* cText = text.c_str();
```

```
std::string value = "abc";
```

```
value.data()[0] = 'A'; // C++17 이후 수정 가능 data()
```



11.8 순차 컨테이너 선택

- 단일 연산의 이론적 복잡도만으로 결정하지 않음
- 원소 수, 접근 패턴, 메모리 배치 확인
- 반복자 안정성과 외부 API 연결도 고려
- 실제 성능은 데이터로 측정

요구 사항	우선 검토할 타입
컴파일 시점 고정 크기	<code>std::array</code>
가변 크기, 빠른 순회, 임의 접근	<code>std::vector</code>
앞과 뒤의 빈번한 삽입과 제거	<code>std::deque</code>
위치를 알고 있는 중간 삽입과 참조 안정성	<code>std::list</code>
앞 방향 순회만 필요한 최소 연결 구조	<code>std::forward_list</code>
가변 길이 문자 시퀀스	<code>std::string</code>

가변 크기 순차 데이터 → 먼저 **vector** 검토
vector가 충족하지 못하는 구체 요구 → 다른 컨테이너 선택



11.8.1 고정 크기와 가변 크기

- 크기가 컴파일 시점에 정해지고 바뀌지 않으면 `std::array`
- 원소 수가 실행 중 결정되거나 증가·감소하면 `std::vector`
- 최대 크기만 정해져 있다고 항상 `std::array`를 사용할 필요는 없음
- 실제 원소 수를 따로 관리해야 하면 `vector + reserve()`가 단순할 수 있음

```
std::array<float, 3> position{;
```

```
std::vector<int> scores;  
scores.reserve(3);
```

상황	선택
항상 좌표 3개	<code>std::array<float, 3></code>
실행 중 추가되는 점수 목록	<code>std::vector<int></code>
최대 100개지만 실제 개수 변동	<code>std::vector<T> + reserve(100)</code>



- 연속 메모리가 필요하면 array, vector, string
- 그래픽, 오디오, 네트워크, 파일 API에 버퍼 전달할 때 중요
- 순차 처리에서도 캐시 지역성에 유리
- 연결 리스트는 원소 이동은 적지만 포인터 추적과 개별 할당 비용 발생

```
std::vector<float> samples;
```

```
ProcessAudio(samples.data(), samples.size());
```

연속 저장

[0][1][2][3][4]

노드 저장

[0] -> [1] -> [2] -> [3]

- 실제 성능은 자료 크기와 접근 패턴으로 측정



- 끝에서만 추가·제거하면 `std::vector`
- 앞과 뒤 양쪽에서 작업하면 `std::deque`
- 위치 반복자를 이미 가지고 있고 참조 안정성이 중요하면 `std::list`
- 원소 순서가 중요하지 않으면 벡터에서 빠른 제거 방식 가능
- 순서가 의미 있는 데이터에는 사용할 수 없음

```
void UnorderedErase(std::vector<int>& values, std::size_t index)
{
    if (index >= values.size())
    {
        return;
    }

    values[index] = std::move(values.back());
    values.pop_back();
}
```

순서 유지 제거 → 뒤쪽 원소 이동

순서 무시 제거 → 마지막 원소와 교체 후 `pop_back`



- 인덱스로 자주 접근하면 array, vector, deque
- 세 컨테이너는 임의 접근 반복자 제공
- list, forward_list에서 n번째 접근은 n에 비례
- 인덱스 접근이 필요한 구조를 연결 리스트로 만들면 불리할 수 있음
- 정렬 알고리즘 선택도 반복자 범주에 영향

```
std::vector<int> values{4, 1, 3, 2};  
std::sort(values.begin(), values.end());
```

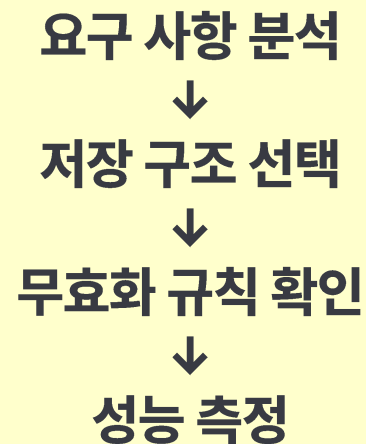
```
std::list<int> nodes{4, 1, 3, 2};  
nodes.sort();
```

컨테이너	인덱스 접근	정렬 방식
vector	빠름	std::sort()
deque	빠름	std::sort()
array	빠름	std::sort()
list	없음	list::sort()



11.8.5 컨테이너 선택 기준

- 원소 수가 고정인지 가변인지 확인
- 연속 메모리가 필요한지 확인
- 인덱스 접근 빈도 확인
- 삽입과 제거 위치 확인
- 반복자, 포인터, 참조 안정성 확인
- 원소당 메모리 부가 비용 확인
- 외부 API 전달 여부 확인
- 예상이 아니라 실제 데이터로 성능 측정



기본 판단	설명
먼저 vector 검토	대부분의 가변 순차 데이터에 적합
다른 컨테이너 선택	벡터가 충족하지 못하는 요구가 분명할 때
성능 판단	실제 데이터와 접근 패턴으로 확인



- 표준 라이브러리는 자료 구조와 알고리즘을 템플릿과 반복자로 연결
- 컨테이너는 원소 저장과 수명 관리
- 반복자는 원소 위치와 범위 표현
- 알고리즘은 반복자 범위에 작업 적용
- 함수 객체와 람다를 비교와 판정 규칙 전달
- STL 전체가 아니라 컨테이너·반복자·알고리즘 중심 설계에서 출발
- 표준 컨테이너는 공통 이름을 제공하지만 저장 구조에 따라 기능과 비용이 다름
- `size()`, `empty()`, `begin()`, `end()`는 여러 컨테이너에서 사용
- 인덱스 접근, 뒤쪽 접근, 역방향 반복, `data()`는 모든 순차 컨테이너에 공통이 아님





- 반복자 무효화는 컨테이너 변경 뒤 기존 위치 객체 사용 가능 여부를 결정
- `std::array`는 고정 크기와 연속 메모리 제공
- `std::vector`는 연속 메모리, 가변 크기, 빠른 임의 접근 제공
- `reserve()`는 용량 확보, `resize()`는 원소 수 변경
- `std::deque`는 양쪽 끝 삽입과 제거가 효율적
- `std::list`와 `std::forward_list`는 노드 기반 연결 구조
- 연결 리스트는 삽입 위치를 알고 있고 참조 안정성이 중요할 때 검토
- `std::string`은 연속 문자 데이터를 소유하고 문자열 조작 기능 제공
- 숫자 변환은 `stoi()` 계열과 `from_chars()`를 상황에 맞게 선택
- 가변 크기 순차 컨테이너의 기본 선택은 `std::vector`
- 벡터가 충족하지 못하는 구체 요구가 있을 때 다른 컨테이너 선택

